

TypeFly: Low-Latency Drone Planning With Large Language Models

Guojun Chen , Xiaojing Yu, Neiwen Ling, and Lin Zhong, *Fellow, IEEE*

Abstract—Recent advancements in robot planning using large language models (LLMs) have demonstrated significant potential, primarily due to LLMs’ capabilities to understand natural language commands and generate executable plans in various languages. However, in time-sensitive and interactive applications involving mobile robots, particularly drones, the sequential token generation process inherent to LLMs introduces substantial latency, i.e. response time, during the control plan generation. In this paper, we present a system called TypeFly that tackles this latency problem using a combination of a novel programming language called MiniSpec and its runtime to reduce both the response time and generation time for the robot plan. That is, instead of asking an LLM to write a program (robotic plan) in the popular but verbose Python, TypeFly gets it to do it in MiniSpec specially designed for token efficiency and stream interpreting. Using a set of challenging drone tasks, we show that design choices made by TypeFly can reduce the average response time to 74% compared to existing works and provide a more consistent user experience, enabling responsive and intelligent LLM-based drone control.

Index Terms—Large language models, drones, command and control systems.

I. INTRODUCTION

MOBILE robots, especially drones, have seen growing applications in both personal and public domains. Recently, the robotics community has leveraged the remarkable capabilities of large language models (LLMs) and their multi-modal counterparts (Vision Language Models) for understanding task descriptions in natural language and generating plans in the form of predefined function calls or Python programs [7], [13], [22], [34], [35], [37], beating traditional robotic control methods. In these applications, the user experience heavily depends on how quickly the robot can start to act after receiving the user’s commands and complete the task [25]. However, LLMs generate tokens one after another in a sequential manner, which creates a fundamental bottleneck in their application to robot control systems. This sequential processing means that the time required to generate a complete output plan increases linearly with the length of that plan. For robotic applications, where real-time responsiveness is often crucial, this latency poses significant

challenges for both the initial response time (how long it takes for the robot to begin executing an action) and the overall completion time of the task. Moreover, this limitation becomes particularly pronounced in scenarios requiring longer action sequences or more detailed planning steps. Thus we need a solution that reduces the response time and simplifies the plan.

This paper presents TypeFly, an end-to-end system that enables an LLM to efficiently accomplish complicated task planning with low response time, as described by Fig. 1. TypeFly introduces a specialized programming language called MiniSpec paired with its stream interpreting runtime. This approach generates drone plans more efficiently than conventional choices like Python and Planning Domain Definition Language (PDDL) [12], requiring fewer tokens and delivering faster responses. Additionally, TypeFly can leverage its probe capability to optimize and simplify plans in certain scenarios.

As illustrated by Fig. 1, a TypeFly user provides a task description in English ①; then the Prompt Generator service combines the task description and scene description (generated by Vision Encoder based on video stream from the drone, in English) into a Planning prompt ② and send it to the Remote LLM ③. The Remote LLM responds, in streaming generation mode, with a plan, which is interpreted by the MiniSpec Runtime ④ and carried out by the drone ⑤. During the interpretation and execution of the plan, the MiniSpec Runtime may engage the Remote LLM through the Prompt Generator ⑥, using a special system skill probe and MiniSpec’s exceptional handling mechanism, namely replan.

We overview TypeFly in Section IV and describe the design of MiniSpec in Section V, and our prototype implementation in Section VI. In Section VII, we report an experimental evaluation of TypeFly using the prototype and a benchmark of 11 tasks of various levels of complexity. Our evaluation shows that the use of MiniSpec can reduce the response time up to 74% and provide relatively consistent performance (< 1.5 s response time for all tasks) across tasks with different complexities, compared to the existing works. It also shows that the mechanism of probe and MiniSpec exception handling is critical to its capabilities of accomplishing tasks with efficiency. Our evaluation also reveals intriguing limits of TypeFly, especially its use of GPT4, which suggests future work (Section VIII). While the user plays the key role of providing a precise and correct task description in TypeFly, we focus on the systems aspects of TypeFly in this paper and leave its usability out of the scope.

In summary, this paper makes the following contributions:

Received 2 October 2024; revised 15 February 2025; accepted 7 April 2025. Date of publication 18 April 2025; date of current version 6 August 2025. This work was supported in part by National Science Foundation (NSF) Athena AI Institute under Award 2112562 and in part by Yale University. Recommended for acceptance by Z. Li. (Corresponding author: Guojun Chen.)

The authors are with the Yale University, New Haven, CT 06520 USA (e-mail: guojun.chen@yale.edu).

Digital Object Identifier 10.1109/TMC.2025.3561282

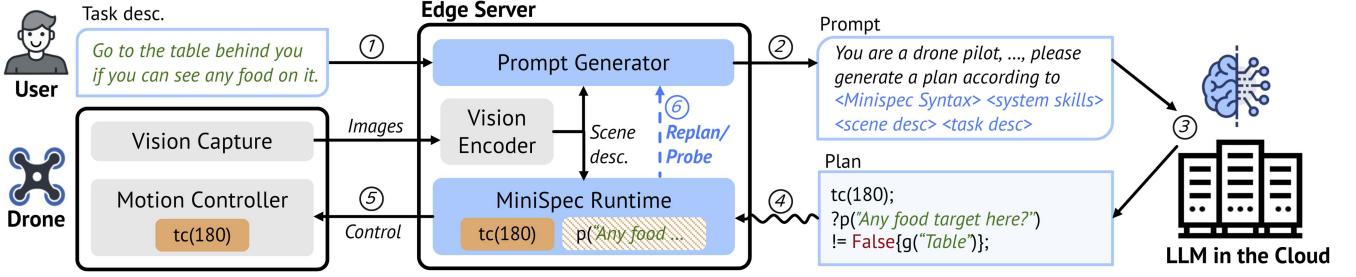


Fig. 1. System overview of TypeFly: An on-premise edge server controls the drone to accomplish a task described by a user, whether a human or a language agent, using natural language. Based on the task description and a scene description by a vision encoder, the LLM writes a program in MiniSpec (Section V-A), called plan. In this example, the plan includes one elementary statement `tc(180)`, which turns the drone by 180-degree, and one composite statement `?p("Any food target here?") != False{g("Table")}`, which moves the drone to a table, depending on whether there is food on it. With *Stream Interpreting* (Section V-C), the drone can start to act on a statement while Remote LLM is still generating the next. TypeFly can deal with syntax errors and unexpected situations through MiniSpec's exception handling mechanism *replan* (Section V-A4). Additionally, using a special skill *probe* (Section V-B), TypeFly can engage the LLM during the plan execution.

- **MiniSpec**, a small programming language designed for LLMs to generate robotic task plans efficiently. MiniSpec features a special skill *probe* to simplify the logic and improve the effectiveness of the plan. MiniSpec also has a special keyword *replan* to handle exceptions such as syntax errors and unexpected situations.
- **TypeFly**, an end-to-end latency-efficient system that flies a drone to accomplish complex tasks described in English.
- **Stream Interpreting**, streaming generation and in-time interpreting execution for LLM-powered drone control, which significantly reduces the response time of the drone. Response time refers to the duration from the user query to the first action of the drone. To the best of our knowledge, we are also the first to use this generation-execution mode in robotic control.

The source code of TypeFly is openly available at <https://github.com/typefly/TypeFly>.

II. BACKGROUND

The capabilities of LLMs have dramatically increased with their growing size. GPT4, one of the largest models and best general-purpose LLMs, excels particularly in reasoning for math, algorithms, and code generation [1]. Serving large models like GPT4 requires massive computational resources, typically involving numerous network-connected GPU servers in data centers, so the most capable LLMs are usually available as cloud-based services. Despite intensive research on training or fine-tuning smaller LLMs for edge-based deployment, these models still lag far behind the capabilities of cloud-based LLMs. For example, according to [41], GPT4 shows an unprecedented 94% Pass@1 rate on the HumanEval [3] dataset, as a comparison, the best local deployable model Code Llama [28] only achieves a 62% Pass@1 rate. Therefore, in this work, we opt for cloud-based GPT4 in designing and implementing TypeFly, to tap the latest capabilities of LLMs. Nevertheless, much of TypeFly's design is agnostic of where the LLM is served, cloud or edge.

a) Latency: The latency of using a remote LLM such as GPT4 consists of that from both the network and computation. At the time of this writing, the computation latency is larger

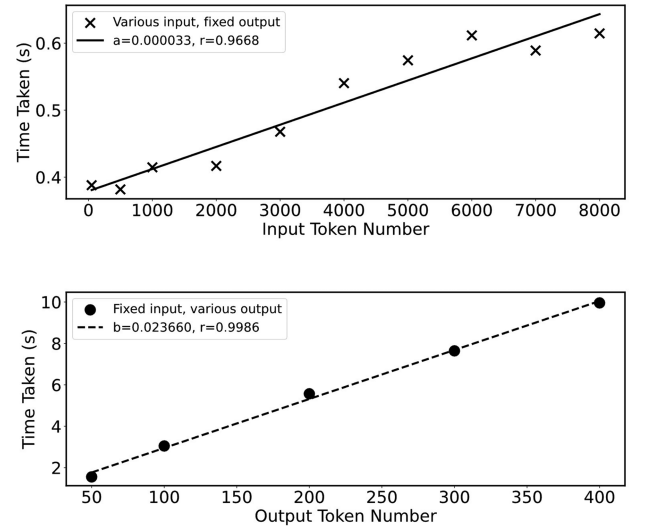


Fig. 2. GPT-4 API latency varies with input and output token counts (avg. of 10 runs). The top figure shows latency vs. input tokens (fixed output), while the bottom varies output tokens (fixed input). Trend lines suggest $b \approx 717a$, which means generating output tokens remains significantly slower - over 700× slower - than processing input. Measurements were taken on March 5, 2025, using gpt4 model.

than the network's (10 s of milliseconds) by at least an order of magnitude for only 5 output tokens. Our experience and measurement indicate that the latency of using GPT4 can be closely approximated with the following formula:

$$latency = a \cdot N_p + b \cdot N_o + c$$

where N_p and N_o are the number of input and output tokens, respectively. They can be measured by the length of the encoded input (prompt) and output text, using the OpenAI tiktoken tokenizer [24]. c is the network latency and service schedule delay. As shown in Fig. 2, $b \approx 2800a$ during our measurement. This implies that the number of output tokens has a major impact on the total latency, provided that the count of input tokens is not significantly (more than 2800 times) larger than that of the output tokens. Based on this finding, TypeFly employs a long-prompt short-output strategy to minimize the required token number for planning (detailed in Section VI).

b) Response Mode: LLMs usually support two modes for generating responses: batch generation and streaming generation. In batch generation, the LLM returns the entire response at once. This can be useful for short, concise answers or when the complete context is needed for further processing. However, it can be less responsive for longer outputs as the user must wait until the entire response is generated. In streaming generation, the LLM generates the response incrementally, sending parts of the response to the user as they are generated. This creates the effect of a continuous, flowing output and improves responsiveness as well as user engagement. TypeFly leverages the streaming generation and the design of MiniSpec to reduce the planning response time.

III. RELATED WORK

Related to TypeFly's end goal of commanding a drone with a natural language, there is a rich literature from the robotics community that apply NLP to robotic control, e.g., [2], [5]. TypeFly is one of many systems that have emerged in the past year applying LLMs to robotic control, achieving significantly better natural language understanding and more flexible planning from task descriptions in natural language [7], [13], [14], [19], [29], [32], [34], [35], [37], [42] or formal robotic languages like PDDL [30], [31]. Due to space limitations, we will focus on related systems that leverage LLMs to generate program-based plans that stand out by enabling more complex tasks with advanced condition evaluations and loops beyond sequential logic. TypeFly introduces latency-reducing innovations while retaining the flexibility of program generation methods.

A key innovation of TypeFly is to design a programming language, i.e., MiniSpec, for LLMs to write programs, which reduces the latency and program errors. To the best of our knowledge, TypeFly is the first to leverage a custom language to optimize the response time when facilitating LLMs in robotic planning. In contrast, most related systems use Python [13], [19], [32], [35], [37], due to its extensive corpus on which most public LLMs were trained. Although Python is designed to be concise, it is intended for humans to write programs. As a result, Python programs are not token-efficient. In contrast, MiniSpec is designed for LLMs and would use on average 30% fewer tokens for the same semantics. Moreover, using Python could introduce unexpected use of third-party libraries or infinite loops, which may not be safe. In contrast, MiniSpec forbids infinite loops and no existing LLMs were trained with MiniSpec libraries (yet). One work uses Keyhole Markup Language (KML) [6] language for flight control in a mission planner like QGroundControl [8]. However, this approach only focuses on planning capabilities. And execution must take place after the entire plan has been generated, as it needs to be fully uploaded to the QGroundControl software for implementation.

TypeFly leverages MiniSpec's exception handling mechanism, `replan`, to implement incremental planning (Section V-A4), which is related to but more efficient than full online planning that is widely used in robotics [9], [23]. Online planning generates a plan for each step of the plan based on the current

scene. In contrast, in TypeFly, `replan` is invoked only when it is necessary.

Another key innovation of TypeFly is to engage the LLM in the execution of a plan via the special skill probe (Section V-B). Related, the authors of [33] also involve the LLM throughout the whole execution phase by asking the LLM to generate a new plan after every several actions, which is both slow and expensive. In contrast, TypeFly incorporates the LLM in a novel way: it steps in to provide responses when confronted with human language requests that the vision model cannot comprehend. This significantly enhances the reasoning capabilities and improves the efficiency of task execution to adapt to a dynamic environment.

Related to TypeFly's support of stream interpretation of MiniSpec plans, the LLM Compiler [17] asynchronously executes functions and generates programs by analyzing dependencies between variables. However, Python's complex syntax tree makes syntax fragmentation extremely challenging, limiting the LLM Compiler to simple sequential execution scenarios. In contrast, MiniSpec allows TypeFly to asynchronously plan generation and execution (even in loop conditions), significantly reducing both response and task completion latencies.

IV. SYSTEM OVERVIEW

TypeFly is a low-latency, end-to-end system that leverages LLM to control the drone via a custom program language MiniSpec, enabling it to accomplish complex tasks with quick response efficiently. In designing TypeFly, we aimed to leverage both the edge computing resources and cloud-based LLM capabilities, to successfully handle non-predefined natural language tasks commanded by users while protecting user's privacy.

As shown in Fig. 1, a drone with the motion controller and video capture capability, MiniSpec-related modules running in an on-premise edge server, and a cloud-powered LLM. The drone sends camera capture to the edge server via a WiFi connection. The user, either a human or a language agent, can observe real-time video feedback and provide high-level instructions to the edge server in a natural language to TypeFly ①, called task description. Simultaneously, the vision encoder on the local server provides a scene description API for other modules based on these images.

The edge server plays a central role in generating effective and efficient MiniSpec by calling the LLM and performing interpreting functions for robot control, utilizing two key components. First, the Prompt Generator sends a prompt to the Remote LLM consisting of the syntax of MiniSpec, task description, scene description, and other relevant information and examples about TypeFly to help formulate executable solutions written in MiniSpec that meet the task objectives ②. Second, MiniSpec Runtime, which is responsible for parsing and executing the MiniSpec plan received from the Remote LLM ③. When MiniSpec Runtime receives the generated token from the Remote LLM in streaming mode ④, it will execute the MiniSpec plan immediately after a statement is generated and parsed executable. Simultaneously, MiniSpec Runtime continuously receives the subsequent parts of the plan from Remote LLM.


```

1 <program> ::= { <composite-statement> [';'] | <statement> ';' } % Program structure
2
3 <statement> ::= <function-call> | <variable-assign> | <exception-handling> | <return> % Different statements
4 <function-call> ::= <function-name> ['(' <argument> ')'] % Function call (search an apple): s('apple')
5 <function-name> ::= 's' | 'sa' | 'tc' | 'p' | 'mf' | ... % Function names can only be system skills
6 <argument> ::= <value> { ',' <value> } % Function arguments
7 <variable-assign> ::= <variable> '=' <function-call> % Variable assignment (save in variable 1): _1=s('apple')
8 <exception-handling> ::= 'replan' | 'rp' % Exception handling
9 <return> ::= '-' <value> % Return statement
10
11 <composite-statement> ::= <loop> | <conditional> % Block statements
12 <loop> ::= <int> '{' <program> '}' % Loop structure (loop for 10 times): 10{...}
13 <conditional> ::= '?' <condition> '{' <program> '}' % Condition stmt. (if found apple, then do...): ?s('apple'){...}
14 <condition> ::= <operand> <comparator> <operand> { '&' <condition> | '|' <condition> } % Condition structure
15 <operand> ::= <value> | <function-call> % Operands in conditions
16 <comparator> ::= '>' | '<' | '==' | '!=' % Comparators
17
18 <value> ::= <literal-value> | <variable> % Values in the language
19 <variable> ::= '_' <int> % Variables
20 <literal-value> ::= <int> | <float> | <string> | <bool> % Literal values

```

Listing 1. Backus–Naur Form Syntax of MiniSpec. MiniSpec meets the all criteria for Turing completeness except the infinite loop, which is intentionally forbidden at the language level. Only system skills can be called as a function with MiniSpec, preventing potential breaks to system integrity. `replan` as the exception handling mechanism can be actively inserted into the plan as a statement, enabling recovery from unexpected situations for improving the success rate.

During the plan interpretation and drone control ⑤, the edge services may also engage the Remote LLM through the Prompt Generator ⑥ by executing a special system skill probe for efficient query and using the keyword `replan` in MiniSpec for exceptional handling.

System’s scalability: While we implement the TypeFly design for miniature drones, our TypeFly is portable to other robotic platforms: only some of the system skills (Drone Control and High-level skills) are platform-specific, while the rest of TypeFly, including MiniSpec, Prompt Generator and Vision Encoder, are platform-agnostic.

User’s Role: The user, either human or language agent, interacts with the TypeFly using a natural language. The user provides the task description, receives the log output, and observes the drone’s real-time video capture as the TypeFly carries out the task. Additionally, the user can terminate the execution at any time.

Vision Analysis: LLM vs. VLM: We opt for a combination of LLM and lightweight vision encoders instead of leveraging the latest advancements in visual-language models (VLMs) [7], [15], [27], [42]. Similar combinations have also been adopted by others [13], [20], [32], [37], [40]. Deploying VLMs directly on edge servers incurs significant computational overhead and latency while deploying them in the cloud raises privacy concerns due to potential reverse-engineering of image embeddings. Our strategy reduces the LLM service cost and enhances privacy by only sending language-based descriptions of the environment to the cloud.

V. MINISPEC

A key idea embraced by TypeFly is to design its own language for LLMs to write programs (plans). We eschew Python, a popular choice for related work, for reasons discussed in Section III. Compared to Python, MiniSpec features concise and limited function sets, simplified syntax, and capable as well as predictable logic control. These features enable the *Stream Interpreting* which plays a critical role in reducing the response latency. Additionally, MiniSpec also supports an exception-handling mechanism to handle expected replanning and unexpected errors during the code execution.

A. Language Design

In MiniSpec language, the semantics of the system skills and plan are defined as a MiniSpec program that adheres to the Backus–Naur Form (BNF) syntax specified in Listing 1. MiniSpec plays a crucial role in achieving efficient and safe Remote LLM usage through four key aspects: (1) Concise and Limited Function Calls: MiniSpec ensures low token count in code generation while preventing unexpected function calls, thereby enhancing efficiency and security. (2) Capable and Predictable Logic Control: MiniSpec supports intricate plan generation and prevents infinite loops, offering versatile and reliable logic control. (3) Simplified Syntax for Streaming Execution: MiniSpec facilitates a simplified syntax set enabling custom parsing and a *Streaming Execution* feature to immediately start code execution whenever an executable piece of the plan is received, reducing response time and improving overall responsiveness. And (4) Exception Handling: MiniSpec provides mechanisms for handling both expected replanning and unexpected errors during execution, improving planning efficiency and accuracy. Each of these aspects is discussed in detail below.

1) Concise and Limited Function Calls: We design MiniSpec to ensure plans are created with a few numbers of tokens while adhering to a limited set of available functions. Specifically, each skill has a unique abbreviation `skill_abbr` which is generated automatically based on `skill_name`, e.g., `iv` for `is_visible` and `tc` for `turn_clockwise`. While executing the plan, only valid callable objects (i.e. system skills and MiniSpec keywords) can be executed, preventing potential safety issues and undefined behaviors. The usage of skills in MiniSpec follows the syntax of function calls (Listing 1), where the skill abbreviation corresponds to the `<function – name>` in the function call. It is important to note that fewer characters do not always result in fewer tokens. For example, “desc.” uses two tokens while “description” uses only one, according to OpenAI’s tiktoken tokenizer [24]. However, we found that all two-character words always use a single token. As a result, the abbreviation of each skill is generated to be unique with at most 2 alphabet characters.

2) **Capable and Predictable Logic Control**: MiniSpec supports conditional control and **bounded** iteration (a loop block must have a fixed predetermined count), enabling the composition of intricate task plans while guaranteeing that all operations will terminate within a bounded number of steps. Furthermore, MiniSpec’s syntax supports variable assignment, providing the mechanism for state manipulation. Consequently, while MiniSpec falls short of Turing completeness criteria due to the absence of support for infinite looping constructs, it still possesses substantial computational capabilities.

3) *Simplified Syntax for Stream Interpreting*: The *Stream Interpreting* feature supported by MiniSpec means during the LLM streaming generation, the code execution can start immediately whenever a valid piece of code is received. We achieve this feature by maintaining a compact and simple syntax, which includes only the logic control, variable assignment, and available system skills. MiniSpec code can be easily parsed by our interpreter. This feature allows MiniSpec to support agile robot control with minimal response time, thereby providing a more engaging and interactive user experience, and being effective for real-time applications. The detail of *Stream Interpreting* will be introduced in V-C.

4) *Exception Handling*: MiniSpec supports exception handling with a special keyword `replan`, which can be used in 3 ways: (1) Remote LLM can use `replan` as a function in the generated plan to improve the planning efficiency and accuracy in a dynamic environment; (2) Users can define custom logic to trigger `replan` when certain actions are likely to cause significant scene changes. For example, `replan` can be triggered if the drone rotates more than 180 degrees or moves forward over 10 meters, as such actions likely indicate a substantial change in the scene. (3) Whenever a syntax error is found during parsing or a mechanical fault occurs during execution, `replan` will be triggered by the interpreter to find a potential fix. Once `replan` is encountered in the plan, triggered by the interpreter, or thrown by system skills, the execution of the plan immediately stops and signals the Prompt Generator to send a new planning prompt to the Remote LLM to regenerate a new plan.

Regarding semantic errors (such as directional mismatches between commanded and intended movements), we acknowledge this is currently beyond TypeFly’s scope. Our work primarily focuses on optimizing code generation latency to enable real-time robot control. While semantic verification is crucial for robust operation, it would require either sophisticated simulation environments or complex verification methods, which we view as valuable future research directions that could complement our current framework.

To further illustrate the use of `replan` when the Remote LLM actively uses it in generated plans, consider the following example: Given the task “Turn around and go to the apple” (one used in evaluation Section VII), a plan generated without `replan` might be

If there is an obstacle between the drone and the apple, the `goto('apple')` command will fail, causing the task to fail. With our design, the Remote LLM can first generate a plan:

Listing 1.

```
l turn_cw(180);goto('apple')
```

```
l turn_cw(180);replan
```

The `replan` will then generate new plans to guide the drone around the obstacle and reach the apple based on the updated scene, such as

```
l move_left(50);goto('apple')
```

When a task involves an environment that is not fully known at the initial planning stage, generating the entire plan at once with the initial scene description may not be efficient or accurate. The `replan` function allows the Remote LLM to reason about potential scene changes in a multi-step task, better adapting to environmental changes and avoiding the overhead of generating unnecessary plans in advance compared to single-pass offline planning methods. Importantly, it is invoked only when necessary, unlike the fully online method where updates are made at every step [9], [23], striking a good balance between efficiency and usability.

B. System Skills

A system skill is a MiniSpec callable object representing the system’s capability. It is typically created by humans and provided to the LLM for use in the plan. Toward the efficiency goal, we start with the best practices available from the literature such as [19] and [35] to design a hierarchical skillset including low-level and high-level skills: low-level skills correspond to callable functions supported by the system, while high-level ones are constructed with other skills and logic controls. All system skills are blocking and return when their action finishes.

Listing 2 shows examples of skill implementation in TypeFly. Skill comes with an expressive name and a `desc` (i.e. description), which help the LLM to understand the functionality. A low-level skill has the most primitive behavior: a single callable function with arguments as supported by the system, e.g., line 3 of Listing 2. However, the behavior of high-level skill, as shown in line 15 of Listing 2, includes a MiniSpec program as a string. Moreover, unlike low-level skills, a high-level skill does not take any arguments explicitly; instead, it will infer the actual arguments from the positional arguments in its MiniSpec definition.

1) *Low-level skills*: TypeFly’s low-level skills include fundamental Drone Control, Vision Tool, User Interface, and one special skill. Assuming all robotic platforms are equipped with a camera, the only platform-specific skills are Drone Control ones. The *Drone Control* skills are basic drone maneuvers that are supported by most drone platforms, such as `move_forward` and `turn_cw`. *Vision Tool* skills offer object-specific details like the target objects’ location, dimension, and color, which are

```

1 LowLevelSkillItem(
2     name="turn_cw", #abbr: tc
3     callable=self.drone.turn_cw,
4     desc="Rotate drone clockwise by certain degrees",
5     args=[SkillArg("deg", int)])
6
7 LowLevelSkillItem(
8     name="is visible", #abbr: iv
9     callable=self.vision.is_visible,
10    desc="Check the visibility of target object",
11    args=[SkillArg("object_name", str)])
12
13 HighLevelSkillItem(
14     name="scan", #abbr: s
15     definition="8{?iv($1)==True{->True}tc(45)}->False",
16     desc="Rotate the drone to find a target")

```

Listing 2. Examples of MiniSpec skill implementation. A skill has a name, an auto-generated abbreviated name, and a description. Low-level skills have callable and args while high-level ones have MiniSpec definitions.

supported by popular computer vision tools such as YOLO. The *User Interface* skills show text or image feedback to the user.

MiniSpec features a specialized skill probe that enhances TypeFly’s capability and efficiency by strategically engaging the Remote LLM during plan execution. The invocation of probe follows three key principles: 1) Runtime Knowledge-Dependent Decisions: probe is invoked when the plan requires dynamic reasoning about the environment that can only be determined during execution. When called, it generates a targeted prompt containing both the `question` (passed as an argument) and the current scene description, ensuring decisions are made with the latest environmental context. 2) Constrained Response Format: To maintain execution efficiency, probe enforces a minimal response format, typically limiting the Remote LLM’s output to 1–2 tokens. This constraint ensures quick decision-making while still leveraging the Remote LLM’s reasoning capabilities without the overhead of online planning. 3) Conditional Branch Optimization: probe is particularly valuable for handling conditional logic that would otherwise require exhaustive enumeration. For example, given the task “Go to the edible object behind you”, probe enables efficient runtime decisions through conditional checks like “Anyedibletargethere?”, as shown in the top of Listing 3. This approach adapts dynamically to the environment, unlike the alternative shown at the bottom of Listing 3, which inefficiently lists all possible edible objects and searches them sequentially. The decision to use probe is driven by the trade-off between plan complexity and execution efficiency. It is preferred when: The decision requires environmental information only available at runtime; The alternative would require an exhaustive enumeration of possibilities; Quick, binary, or categorical decisions are needed during the execution; The plan needs to adapt to varying environmental conditions; This strategic use of probe results in more concise plans, complete coverage of possibilities, and more consistent execution times regardless of the specific environmental state.

2) *High-level skills*: TypeFly’s skillset selectively includes a small number of high-level skills to improve the cost efficiency of plans and to simplify the programming job of the Remote LLM. Low-level skills constitute the entire capability of TypeFly. In theory, the Remote LLM can

```

1 8{ 1=p('Any edible target here?');_1!=True{g(_1)->True}tc
  (45)}->False

1 ?s('apple')==True{g('apple')->True}
2 ?s('cake')==True{g('cake')->True}
3 ?s('sandwich')==True{g('sandwich')->True}
4 ?s('orange')==True{g('orange')->True}
5 ->False

```

Listing 3. Example plans produced by the LLM (GPT-4) with (top) and without (bottom) probe skill for the task “Find something edible”. s, p, and g are the abbreviations for sweeping, probe, and goto respectively.

produce a plan solely based on them. Such a plan, however, is verbose and as a result, not cost-efficient. On the other extreme, one could provide a large and semantically rich set of high-level skills, converting all previously successful plans into high-level skills. Because the entire skillset is included in a Planning prompt, a large skillset with numerous high-level skills increases the cost of Remote LLM service. Moreover, our experience shows that adding too many high-level skills will cause the Remote LLM to produce plans with only high-level skills and result in incorrect plans. Balancing between the above two extremes, TypeFly includes a small number of high-level skills. They are `scan(obj_name)`, `scan_abstract(description)`, `orienting(obj_name)`, and `goto(obj_name)`. These skills are widely used by most tasks to find and interact with target objects, having them in the high-level skillset can simplify and improve the quality of the plan with little overhead in the planning prompt.

Listing 4 shows a high-level skill `scan` in MiniSpec along with an equivalent Python code representation. `scan` rotates the drone for 360° in 8 steps, checking if the given object is present. Whenever the target object is spotted during the loop, the skill returns `True`; otherwise, the drone is rotated clockwise by 45 degrees. If the loop completes, the skill returns `False`. In particular, the above skill definition in MiniSpec takes just 18 tokens, while its Python equivalent requires 41 tokens, as measured by the OpenAI tiktoken tokenizer [24].

```

1 8{?iv($1)==True{->True}tc(45)}->False

1 def scan(object_name):
2     for i in range(8):
3         if vision_skill.is_visible(object_name) == True:
4             return True
5         drone_skill.turn_ccw(45)
6     return False

```

Listing 4. Semantic definition of the high-level skill `scan` in MiniSpec (18 tokens, top block) and its Python equivalent (41 tokens, bottom block).

C. MiniSpec Runtime

One of the key factors featured by TypeFly to reduce the response time is *Stream Interpreting*, which allows the drone to start action while Remote LLM is still generating the plan. As a comparison, starting execution after the whole plan is received, used by most works, is called Normal Interpreting here.

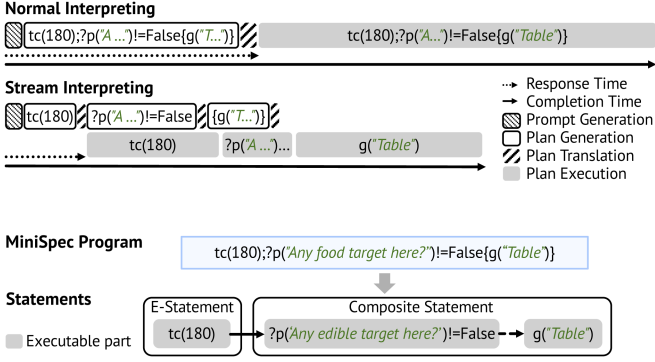


Fig. 3. Normal versus Stream Interpreting of a MiniSpec plan and MiniSpec parsing. In Normal Interpreting, TypeFly waits for the whole plan to be received from Remote LLM to start translation and execution. The response time is highly related to the length of the plan and makes the drone less responsive when the plan is long. In *Stream Interpreting*, the response time is reduced to receiving the first executable part of the plan. The bottom half shows the executable part for different types of statements. Note that the network latency is omitted in the figure.

a) *Stream vs. Normal Interpreting*: Below we introduce how *Stream Interpreting* can reduce the response time and improve user experience in real-time applications. As illustrated in Fig. 3, with Normal Interpreting, the Prompt Generator feeds the prompt into Remote LLM and the system waits for the whole plan (blue strip) to be received and then starts translation and execution. As a result, the response time varies with the length of plans and can cause the system to be less responsive when instructed to perform complex tasks. In contrast, *Stream Interpreting* treats the plan as a stream, with the design of MiniSpec, our MiniSpec Runtime can easily identify executable statements (blue blocks in Fig. 3 and will be detailed later in this section) and start execution as soon as possible. Such a design reduces the waiting time between the user's command and the robot's action, making the control system more responsive and appropriate for real-time applications. A contemporary work, LLMCompiler [17], employs a similar approach of the prefetching for execution at the task level during LLM decoding to improve latency performance in QA applications, however, the MiniSpec and MiniSpec Runtime support this feature at the language level and provides a wider range of applicability.

Next, we introduce how *Stream Interpreting* is achieved through the combination of MiniSpec language design (introduced in Section V-A) and MiniSpec runtime. The MiniSpec runtime is an interpreter that can simultaneously receive, parse, and execute the MiniSpec plan. MiniSpec runtime has two threads. One preprocessing thread accepts the streamed LLM response, parsing and formatting the MiniSpec code piece into Statements. The other worker thread employs an infinite loop to accept Statements from the first thread through a queue.

A Statement represents a block of MiniSpec code, which can be a function call, a variable assignment, a conditional block, or a loop block. The former two types are Elementary Statements, e.g. `tc(180)` in Fig. 3, and the latter two types are Composite Statements. A MiniSpec program is a sequence of Statements. Whenever an Elementary

Statement is complete or a Composite Statement is partially complete, it's considered executable, and MiniSpec runtime will send it to the worker thread for execution.

For Composite Statements, the parsing and execution are various depending on the type. A condition Statement starts with a condition (with a leading "?"), which can be a combination of intersection and union of sub-conditions. Each sub-condition can involve the comparison between the return value of a skill call and literal values. Once the condition is completely received, the Statement will be sent to the worker thread without waiting for the body block (embraced by "{ }"). While the worker thread is executing the condition, the other thread will continue to append sub-statements into the body block. If the condition results is True, the worker thread will start executing the body block. In rare cases, the body block is still empty after the condition execution. In these cases, the worker thread will wait until the next executable Statement to be appended into the block body.

For Elementary Statements, the process is straightforward. After a complete one is received by the preprocessing thread, it will be sent to the worker thread for execution. The worker thread simply calls the system skill (or does variable assignment) in blocking mode.

Notably, in our application, most Statements take significantly longer time to execute than the time to be generated by LLM and cost by preprocessing.

VI. IMPLEMENTATION

We next provide the necessary information about our implementation of TypeFly on which the evaluation is based.

Hardware: TypeFly uses a commercial-off-the-shelf drone and an in-house edge server as the hardware deployment. The drone, Tello, is made by Ryze Tech as shown in (Fig. 4). It offers Wi-Fi video streaming (at most HD720P/30 frames per second (FPS)) and user-friendly operations through a smartphone application. Moreover, it comes with a Python interface library which provides APIs for both control and video streaming. Our edge server is equipped with a Nvidia RTX4090 GPU, a 16-core Ryzen 7950x CPU, and 64 GB RAM. All system-related software, with the exception of the Remote LLM (OpenAI GPT4), operates on this server and collectively consumes at most 20% of the total available resource.

Software: The TypeFly software comprises approximately 2500 lines of Python code and is structured as follows: (1) A User Interaction Module: TypeFly users interact with the system via a web-based interface, as shown in Fig. 4(d). Through this interface, the user can provide task descriptions, observe real-time video feedback from the drone, and read scene descriptions output by the Vision Encoder. (2) Vision Encoder: The Vision Encoder generates the scene description based on the drone-captured image. Here we opt for YOLOv8 due to its rapid processing speed and its ability to provide precise geometric information, such as the location of objects. (3) Prompt Generator: This part integrates the user's task description with the system description and the current scene description from the Vision Encoder. Then it composes a prompt and sends

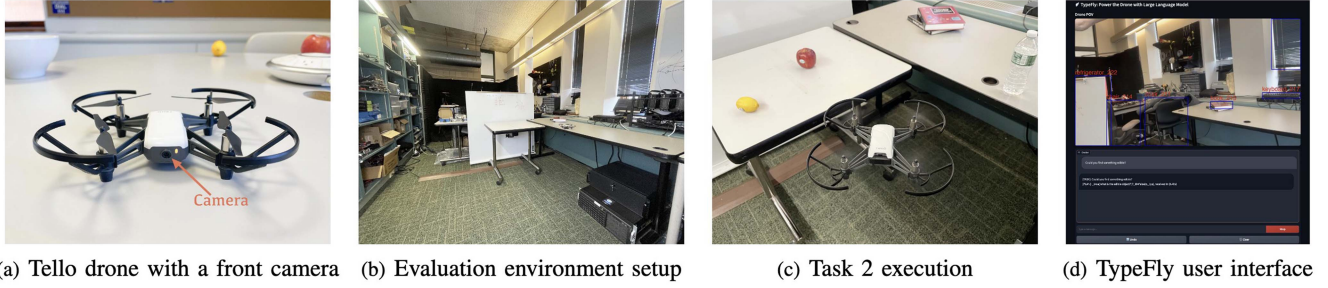


Fig. 4. Evaluation setup and the screenshot of TypeFly interface. We use a cheap off-the-shelf drone with video streaming and programmable control API in our evaluation, showing the potential of TypeFly’s portability for other kinds of robots. The experiments are done inside a typical office area without any external infrastructure except for our edge server and WiFi.

it to Remote LLM (OpenAI GPT4 API) for plan generation. (4) *MiniSpec Runtime*: Detailed in V-C, the *MiniSpec Runtime* functions to interpret and execute the *MiniSpec* plan generated by the Remote LLM.

Here we provide a more detailed design overview of Prompt Generator and Vision Encoder.

Prompt Generator: We opt for a long-prompt short-output strategy instead of generating rich output (e.g. Chain-of-Thought [36] or ReAct [38]) for the use of Remote LLM for two reasons: (1) With our goal of low latency control, we expect the LLM to generate only the necessary tokens to minimize the plan generation time. (2) Given the first reason, in order to improve the planning accuracy, we use few-shot prompting with detailed explanations and guides. The drawback of this design is that, with the current cost model of LLMs, TypeFly would result in a higher cost than a system that uses a smaller prompt. However, we observe that works like [10] have shown the potential of reducing the cost for LLMs service of a big structured prompt by reusing the prompt cache. We believe that the cost of TypeFly will be reduced in the future.

Specifically, the prompt generator will construct a comprehensive prompt that incorporates different essential elements for different purposes. (1) For task planning, the prompt integrates the user’s task description, the current scene information from Vision Encoder, a detailed description of the language’s skill set and syntax, and a list of few-shot examples for demonstrating how *MiniSpec* can be used to formulate executable solutions that meet the task objectives. Moreover, we provide some natural language guidance that aids the LLM in how to use the system skills, how to reason for the task, and the requirements for the output. For example, we provide insight into the proper use of *probe*: only when the required information is missing in the current scene, *probe* should be called with an appropriate question. (2) For exception handling (*replan*), the prompt includes the basic planning prompt introduced above and the history of the previous plan, enabling the LLM to determine the extent of plan execution and generate a subsequent plan accordingly. (3) For *probe*, we opt for a lightweight question and answer (Q&A) prompt including basic rules and two examples for response generation.

Notably, the system skills in the prompt are robot-specific. *Prompt Generator* defines the low-level skills required for a drone with the *DroneWrapper* abstract class, which

can easily be applied to other drone platforms with a Python interface for basic movement control and video streaming. When working with other types of robots, developers only need to design the skillset for the robot and update the planning prompt accordingly.

Vision Encoder: Our Vision Encoder uses YOLOv8 and it generates a list of detectable objects with their name of types and bounding boxes. The object type is used for task reasoning with Remote LLM and the bounding box is necessary for locating and approaching the target object. This kind of information is essential when privacy is concerned, as it is important to avoid sending images to the Remote LLM in such cases. The YOLOv8 service is deployed independently to the other part of the system following a micro-service-like architecture and it can be accessed through gRPC [11] request. While our current implementation uses YOLO [16] for object detection, it can be trivially extended to incorporate additional vision models such as CLIP [26], BLIP [18], YOLO-World [4], in future iterations.

VII. EVALUATION

We evaluate TypeFly with a set of increasingly challenging tasks as summarized by Table I. We seek to answer the following questions:

- What kinds of tasks are feasible vs. infeasible for TypeFly?
- How effective is TypeFly in reducing the response time compared to other solutions?
- When and why would TypeFly fail?
- How effective is *MiniSpec* and *MiniSpec Runtime* in improving TypeFly’s efficiency and user experience?
- How effective is *probe* in improving TypeFly’s efficiency and capability?
- How effective is *replan* in improving TypeFly’s capability?

We choose to evaluate the effectiveness of TypeFly’s key innovations via ablation studies, instead of a direct comparison to related systems, for the following reasons. First, related systems use various robotic platforms (and simulators) in their evaluations. While their software may be open-source, their robotic hardware is not generally available, which makes an apple-to-apple comparison difficult, if possible at all. This is

TABLE I
BENCHMARK TASKS USED IN THE REPORTED EVALUATION

Categories	ID	Task Description	Scene Setup
Basic Planning	1	Go and take a picture of the chair.	a chair in sight
	2	Could you find an apple? If so, go to it.	an apple in the scene
	3	Go to the largest item you can see right now.	a person, apple, and keyboard in sight
LLM in Execution	4	Find something yellow and sweet.	banana and lemon on the table behind the drone
	5	Can you find something for cutting paper on the table? The table is on your left.	table on the left with a pair of scissors
	6	Find a chair and go to the object that is closest to the chair.	a chair on the back with an apple on the chair and a bottle behind the chair
Complex Planning	7	Move up for 1m and check the top of the cabinet, if you see anything red and sweet, take a picture of it. Otherwise, return to the original position.	an apple on top of the cabinet
	8	Can you find something for me to eat? If you can, go for it and return. Otherwise, find and go to something drinkable.	only coke on the left table without any other food
Incremental Planning & Replanning	9	Turn around and go to the apple.	an apple on the table behind the drone with a chair blocking in between
	10	If you can see more than two people behind you, then turn to the tallest one that is behind you.	3 people in sight and 2 other people in the back of the drone
Safety of MiniSpec	11	Turn around in a 45-degree step until you see a person with a cup in hand.	none

We define 5 types of tasks to test TypeFly's capabilities thoroughly. Tasks 1-3 test TypeFly's basic ability to reason and plan; Tasks 4-6 test using Remote LLM in execution to generate offline plans for the undetermined targets; Tasks 7-8 test several more complex planning scenarios; Tasks 9-10 test the TypeFly's ability of exception handling; Last task evaluates the safety of MiniSpec to generate a plan with termination.

particularly true because system skills and prompt engineering are highly specific to the robotic hardware. Second, the key innovations of TypeFly, i.e., MiniSpec and MiniSpec Runtime, are agnostic of the robotic hardware and as a result, their impact can be revealed by proper ablation studies.

A. Setup

Physical Environment: The reported evaluation results were obtained from flying the drone inside an office room, as shown in Fig. 4:(b). The room includes tables, chairs, and a variety of objects. Among these objects, some are directly related to the tasks while others are not. During evaluation, we found, not surprisingly, that a proper lighting condition is crucial to the performance of YOLO and the drone's stability.

Benchmark Tasks: During the evaluation, we tested TypeFly with a list of tasks to test its limit and effectiveness: when it works, when it does not, how efficient the system is? Due to the space limit, we must curate the benchmark tasks such that they demonstrate both the capability and the limit of TypeFly, illuminate the roles of key ideas of TypeFly, such as MiniSpec, probe, and replan, as well as point the direction for future improvement. Guided by these objectives, we select 11 tasks of five categories as summarized by Table I. Importantly, we define a task by task description and scene setup, because performance depends on both. Our benchmark tasks are substantially more complicated than those used by prior work with drones such as [34], [35]; none of them would be supported by prior systems.

Metrics: Given a task, we evaluate the performance of TypeFly with the following metrics.

- whether TypeFly accomplishes the task, i.e., success or not. This is about the semantic correctness of the plan produced by GPT4 and the feasibility of the task with TypeFly.
- the response time it takes TypeFly to generate the initial plan (R-Time). The response time means the duration from the user query to the first action of the drone and it includes

TABLE II
EVALUATION RESULTS OF THE BENCHMARK TASKS BASED ON 10 RUNS OF EACH

ID	Success	Ave./Std.		
		R-Time (s)	C-Time (s)	O. Token #
1	10/10	1.24/0.13	7.83/0.85	7/0
2	10/10	1.28/0.08	12.51/1.29	10/0
3	10/10	1.26/0.10	7.28/0.98	6/0
4	10/10	1.35/0.15	15.84/2.37	21/0
5	10/10	1.19/0.10	11.07/1.37	27/0
6	10/10	1.22/0.12	9.95/1.00	38/0
7	10/10	1.19/0.13	38.00/2.56	40/0
8	10/10	1.32/0.11	47.90/5.86	47.1/0.6
9	7/10	1.10/0.13	15.44/1.90	15/0
10	10/10	1.22/0.13	12.35/1.41	35.0/6.3
11	10/10	1.60/0.13	26.10/3.19	33/0

Most tasks are successfully accomplished by TypeFly. The average response time is relatively independent of the length of the plan due to our *Stream Interpreting* design. Task 9 has several failure cases which will be discussed in §VII-B

a relatively consistent roundtrip network latency (around 50 ms) between the edge server and the cloud-based GPT-4 model.

- the task completion time it takes TypeFly to finish the task (C-time). The total time refers to the duration from the user query to the task completion by the drone.
- the total number of tokens in the output plans (Token #)

B. Overall Performance of TypeFly

Success Rate: Table II presents the metrics of all benchmark tasks. We repeated each task 10 times and reported the average along with the standard deviation (Std). TypeFly succeeded in finishing most tasks, except for several failure cases for Task 9: *Turn around and go to the apple*. For this task, we aim to test TypeFly's capability to generate a correct plan when the engaged scene setup changes. The scene setup includes an apple on the table behind the drone with a chair blocking in between. TypeFly fails to accomplish this task in 3 out of 10 runs. The failure is

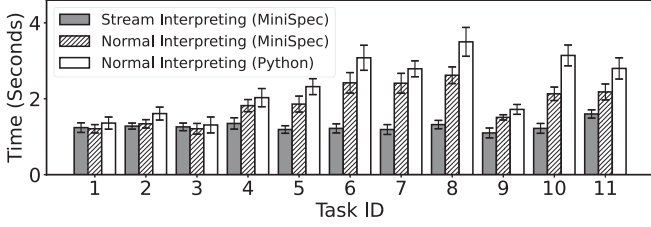


Fig. 5. Response time comparison of TypeFly using MiniSpec with *Stream Interpreting*, MiniSpec with Normal Interpreting, and Python with Normal Interpreting. The Python plan adheres to the same logic as the MiniSpec plan. Using MiniSpec results in at most 32% reduction in response time and further employ *Stream Interpreting* can achieve up to 62% response time reduction as well as provide a more consistent performance when compared with the Python baseline.

rooted in that TypeFly cannot well estimate the geometric size of the obstacle (the chair). It falsely assumes the apple is reachable after the drone moves leftwards a little bit. However, the edge of the chair can still block the drone from reaching the apple.

Plan Length and Response Time: The length of the generated plan for the benchmark tasks vary from 6 (task 3) to 47 (task 8). It is obvious that the length is related to the task complexity. The response time includes the network round trip time (RTT) to GPT4 and the latency of GPT4 generation. Because the network RTT is on the level of 10 s milliseconds, the response time is dominated by GPT4’s inference latency. Not surprisingly, in the Normal Interpreting mode, the response time is highly correlated with the plan length: generally, shorter plans experience shorter response time. However, with *Stream Interpreting*, the response time is similar for all tasks due to the design of MiniSpec and MiniSpec Runtime, which highlights our efficient target.

Task Completion Time: The task completion time of a plan is also roughly related to its token number. The most complex task (task 8) takes the longest time to finish. Environmental variation and the possible use of a bounded loop reduce the correlation between plan length and task completion time.

C. TypeFly’s Generation Efficiency

We compare TypeFly’s plan response time with FLTRNN [39], which is a Python-based solution, and LLM+P [21] which is a PDDL-based solution. Specifically, PDDL (Planning Domain Definition Language) is a standardized language used in automated planning and scheduling, particularly in robotics and AI. It was developed to create a common format for describing planning problems and domains. As shown in Fig. 6, compared to FLTRNN and LLM+P, TypeFly achieves significant reductions in response time: an average reduction of 46% versus FLTRNN and 74% versus LLM+P, with maximum reductions of 61% and 82% respectively.

D. Effectiveness of TypeFly Component

1) **Effectiveness of MiniSpec:** MiniSpec serves as the key to improving task planning efficiency by minimizing both the response time and the number of output tokens required

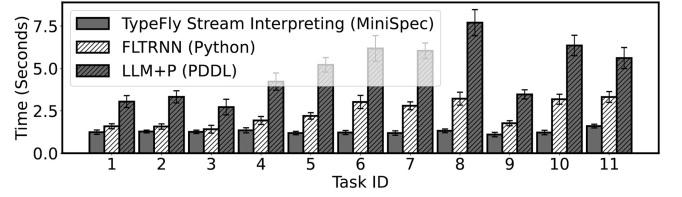


Fig. 6. Response time comparison of TypeFly with FLTRNN and LLM+P. FLTRNN performed similarly to our Python baseline in terms of speed. However, LLM+P was significantly slower since it generates plans in PDDL format, which requires many more tokens.

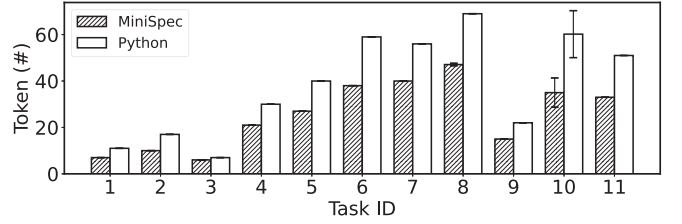


Fig. 7. Token number comparison of the system using MiniSpec vs. using Python to generate the plans. The Python implementation also features prompt engineering and system skill design. Using MiniSpec results in up to a 42% reduction in token numbers and an average reduction of 34%.

for a given plan. As illustrated in Fig. 5, the MiniSpec itself with Normal Interpreting achieves at most a 32% reduction in response time. Moreover, with *Stream Interpreting*, MiniSpec can save up to 62% response time, thereby significantly improving the robot’s responsiveness in real-time and interactive applications. This reduction in response time also leads to more consistent performance across tasks of varying complexity, ultimately enhancing the user experience. In Fig. 7, we also show the complete plan token numbers for the whole task list when using MiniSpec and Python. On average, MiniSpec reduces the output token number by 34%.

2) **Effectiveness of probe:** The probe plays a significant role in improving the system’s efficiency, both plan generation and execution. We evaluated the absence of probe on tasks 4, 5, 6, 7, and 8, since they are related to abstract object identification in unseen environments. For tasks 4, 6, and 7, our system fails to generate a feasible plan without probe since the planner cannot determine the target object at the planning stage. For task 5, remote LLM happens to generate a plan to find “scissors” which is a reasonable target for cutting paper, however, this is not reliable. For task 8, the generated plan loops over several edible and drinkable targets, making the plan very long and inefficient.

In Fig. 8, the advantages of integrating Remote LLM during execution are evident. By doing so, there is a significant reduction in output token length, plan request latency, and overall task completion time. This is because when equipped with the probe function, Prompt Generator can apply the knowledge base of the Remote LLM to a dynamic environment more efficiently with offline planning.

3) **Effectiveness of Exception Handling:** We conduct a similar evaluation for tasks 9 and 10 without `replan` to evaluate the effectiveness of our exception handling. The results reported in Table III show that `replan` is crucial in improving the success

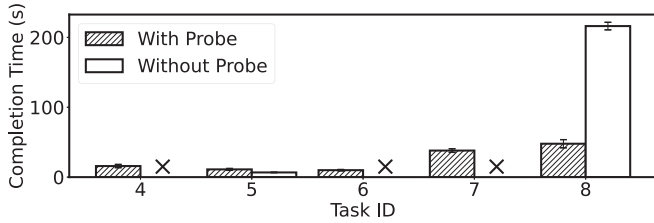


Fig. 8. Task Completion time of the plan generated by the system with/without the skill probe for the five tasks. For tasks 4, 6, and 7 without probe, TypeFly fails to fulfill the task. In task 5, the system works coincidentally by directly searching for scissors since it's a reasonable target for cutting paper. Without probe, task 8 results in a long plan and long completion time with enumeration for every edible and drinkable object.

TABLE III
EVALUATION OF TASKS 9 AND 10 WITH/WITHOUT `replan`

ID	Success	Ave./Std.		
		R-Time (s)	C-Time (s)	O. Token #
With replan				
9	7/10	1.10/0.13	15.44/1.90	15/0
10	10/10	1.22/0.13	12.35/1.41	35.0/6.3
Without replan				
9	0/10	-	-	-
10	8/10	1.19/0.21	13.82/1.53	27.2/0.8

For task 9, without knowing the potential obstacles the task fails all the runs; For task 10, the Remote LLM occasionally confuses the foreground people with the actual targets and mistakenly believes that the condition is met. The task will fail without `replan` when such a mistake happens.

rate and efficiency for tasks related to the unseen environment. Below we dive into each task and reveal how `replan` helps.

Task 9: As we discussed before, even with scene-based incremental planning, TypeFly fails to accomplish this task in 3 out of 10 runs due to the lack of geometric understanding of the environment. Without `replan`, TypeFly is unable to know the obstacle (the chair) at the initial planning stage and fails to accomplish the task in all 10 runs.

Task 10: This task is conducted in a room with 3 people in sight and 2 other people in the back of the drone. TypeFly fails to accomplish this task in 2 out of 10 runs. The failure is rooted in that Remote LLM does not understand the potential scene changes when actions like “turn around” are not explicitly mentioned in the task description. It falsely assumes the 3 people in the original scene description are the same set of visible people after the drone turns 180 degrees. Indeed, if we modify the scene setup such that there is nobody in the original scene description, TypeFly will produce a correct plan. Since task 10 is all about people originally behind the drone, whether there were people originally before the drone should not have mattered. With `replan`, after the drone turns 180 degrees, TypeFly can generate a correct plan by re-counting the people and determining the tallest one from the new scene description.

VIII. CONCLUDING REMARKS

In this paper, we introduce TypeFly, a system that allows drones to accomplish low-latency drone control with natural

language commands. TypeFly achieves this through a synthesis of edge-based vision intelligence, innovative custom programming language MiniSpec, and prompt engineering with LLMs. Through a series of progressively complex drone tasks, TypeFly demonstrates its ability to substantially reduce the system's response time and provide a more consistent user experience with efficient task completion. Crucially, the integration of probe and the exception handling mechanism (`replan`) of MiniSpec in TypeFly markedly enhances the chance of accomplishing complicated tasks. Our experience with TypeFly also reveals numerous challenges to tackle as well opportunities to explore in future work, which we will briefly discuss below.

The tasks involved in our evaluation can mostly be accomplished through a simple scene description based on the derived object list and their positions. However, for more complex tasks that require a comprehensive understanding of the scene and environment, TypeFly falls short without historical observation memory and environment modeling. While past works have utilized sensors like Lidar, we aim to explore the drone's spatial modeling and understanding in the future, particularly on lightweight, cost-effective drones without many high-precision sensors.

The interaction method in TypeFly involves sending the complete prompt content to the LLM each time. For different tasks, only the scene description and task description can vary, comprising less than 8% of the total prompt length. Recomputing the repeated content wastes resources and increases latency. Recent approaches, such as using cache to reuse attention states, have effectively reduced latency, especially time-to-first-token [10]. We will explore this further to improve the speed of LLM inference for robot control.

REFERENCES

- [1] J. Achiam et al., “GPT-4 technical report,” 2023, *arXiv:2303.08774*.
- [2] V. Blukis, Y. Terme, E. Niklasson, R. A. Knepper, and Y. Artzi, “Learning to map natural language instructions to physical quadcopter control using simulated flight,” in *Proc. Conf. Robot Learn.*, 2019, pp. 1415–1438.
- [3] M. Chen et al., “Evaluating large language models trained on code,” 2021, *arXiv:2107.03374*.
- [4] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-world: Real-time open-vocabulary object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16901–16911.
- [5] E. M. Craparo, *Natural Language Processing for Unmanned Aerial Vehicle Guidance Interfaces*. Cambridge, MA, USA: MIT Press, 2004.
- [6] Q. Dong, X. Chen, and M. Satyanarayanan, “Creating edge AI from cloud-based LLMs,” in *Proc. 25th Int. Workshop Mobile Comput. Syst. Appl.*, 2024, pp. 8–13.
- [7] D. Driess et al., “PaLM-E: An embodied multimodal language model,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 8469–8488.
- [8] DroneCode, “QGroundControl: Intuitive and powerful ground control station for the MAVLink protocol,” 2023. [Online]. Available: <https://qgroundcontrol.com/>
- [9] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox, “Online replanning in belief space for partially observable task and motion problems,” in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 5678–5684.
- [10] I. Gim, G. Chen, S.-S. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt cache: Modular attention reuse for low-latency inference,” in *Proc. Mach. Learn. Syst.*, 2024, pp. 325–338.
- [11] Google, “gRPC,” 2023. [Online]. Available: <https://github.com/grpc/grpc>
- [12] M. Helmert, “Concise finite-domain representations for PDDL planning tasks,” *Artif. Intell.*, vol. 173, pp. 503–535, 2009.
- [13] S. Huang, Z. Jiang, H. Dong, Y. Qiao, P. Gao, and H. Li, “Instruct2Act: Mapping multi-modality instructions to robotic actions with large language model,” 2023, *arXiv:2305.11176*.

- [14] W. Huang et al., “Grounded decoding: Guiding text generation with grounded models for robot control,” 2023, *arXiv:2303.00855*.
- [15] Y. Jiang et al., “VIMA: General robot manipulation with multimodal prompts,” in *Proc. 40th Int. Conf. Mach. Learn.*, 2023.
- [16] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [17] S. Kim et al., “An LLM compiler for parallel function calling,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 24370–24391.
- [18] J. Li, D. Li, C. Xiong, and S. Hoi, “BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” 2022, *arXiv:2201.12086*.
- [19] J. Liang et al., “Code as policies: Language model programs for embodied control,” in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 9493–9500.
- [20] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, “Text2Motion: From natural language instructions to feasible plans,” *Auton. Robots*, vol. 47, pp. 1345–1365, 2023.
- [21] B. Liu et al., “LLM+P: Empowering large language models with optimal planning proficiency,” 2023, *arXiv:2304.11477*.
- [22] S. Mirchandani et al., “Large language models as general pattern machines,” in *Proc. Conf. Robot Learn.*, 2023, pp. 2498–2518.
- [23] T. Nageli, L. Meier, A. Domahidi, J. Alonso-Mora, and O. Hilliges, “Real-time planning for automated multi-view drone cinematography,” *ACM Trans. Graph.*, vol. 36, pp. 1–10, 2017.
- [24] OpenAI, “tiktoken,” 2023. [Online]. Available: <https://github.com/openai/tiktoken>
- [25] H. Pelikan and E. Hofstetter, “Managing delays in human-robot interaction,” *ACM Trans. Comput.-Hum. Interact.*, vol. 30, pp. 1–42, 2023.
- [26] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [27] S. Reed et al., “A generalist agent,” *Trans. Mach. Learn. Res.*, 2022. [Online]. Available: <https://openreview.net/forum?id=1ikK0kHjv>
- [28] B. Roziere et al., “Code Llama: Open foundation models for code,” 2023, *arXiv:2308.12950*.
- [29] D. Shah et al., “LM-NAV: Robotic navigation with large pre-trained models of language, vision, and action,” in *Proc. Conf. Robot Learn.*, 2023, pp. 492–504.
- [30] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. Kaelbling, and M. Katz, “Generalized planning in PDDL domains with pretrained large language models,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 20256–20264.
- [31] T. Silver, S. Dan, K. Srinivas, J. Tenenbaum, L. Kaelbling, and M. Katz, “PDDL planning with pretrained large language models,” in *Proc. AAAI Conf. Artif. Intell.*, 2024.
- [32] I. Singh et al., “ProgPrompt: Generating situated robot task plans using large language models,” in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 11523–11530.
- [33] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “LLM-Planner: Few-shot grounded planning for embodied agents with large language models,” in *IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 2998–3009.
- [34] M. L. Tazir, M. Mancas, and T. Dutoit, “From words to flight: Integrating OpenAI ChatGPT with PX4/Gazebo for natural language-based drone control,” in *Proc. Int. Workshop Comput. Sci. Eng.*, 2023, pp. 215–222.
- [35] S. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, “ChatGPT for robotics: Design principles and model abilities,” *IEEE Access*, vol. 12, pp. 55682–55696, 2024.
- [36] B. Wang et al., “Towards understanding chain-of-thought prompting: An empirical study of what matters,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 2717–2739.
- [37] J. Wu et al., “TidyBot: Personalized robot assistance with large language models,” *Auton. Robots*, vol. 47, pp. 1087–1102, 2023.
- [38] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Representations*, 2023.
- [39] J. Zhang et al., “FLTRNN: Faithful long-horizon task planning for robotics with large language models,” in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 6680–6686.
- [40] X. Zhao, M. Li, C. Weber, M. B. Hafez, and S. Wermter, “Chat with the environment: Interactive multimodal perception using large language models,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 3590–3596.
- [41] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language agent tree search unifies reasoning acting and planning in language models,” in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 62138–62160.
- [42] B. Zitkovich et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. Conf. Robot Learn.*, 2023, pp. 2165–2183.



Guojun Chen received the BS degree in computer science from Peking University, Beijing, China, in 2019. He is currently working toward the PhD degree with Yale University. His research is concerned with robotic system design, including hardware, operating system, and low-latency large language model-based agents.



Xiaojing Yu received the BS and PhD degrees in computer science from the University of Science and Technology of China (USTC), Hefei, China, in 2018 and 2023, respectively. She is currently a postdoctoral associate with Yale University. Her research interests include visual intelligence and edge computing.



Neiwen Ling received the PhD degree from the Chinese University of Hong Kong, in 2022. She is currently a postdoctoral associate with Yale University. Her research interests include the intersection of edge computing, machine learning, CPS/IoT, and real-time systems, with a focus on designing time-sensitive AI systems for physical agents. She has received several honors, including the Best Paper Award Finalist and Best Poster Award at ACM SenSys 2022, the Best Artifact Award Runner-Up at ACM MobiCom 2024, and the N2Women Young Researcher Fellowship.



Lin Zhong (Fellow, IEEE) received the BS and MS degrees from Tsinghua University, in 1998, 2000, respectively and the PhD degree from Princeton University, in 2005. He is Joseph C. Tsai professor of computer science with Yale University. From 2005 to 2019, he was with Rice University. At Yale, he leads the Efficient Computing Lab to make computing, communication, and interfacing more efficient and effective. He and his students received the best paper awards from ACM MobileHCI, IEEE PerCom, ACM MobiSys (3), ACM ASPLOS, IEEE QCE and NDSS.

He is a recipient of the NSF CAREER Award, the Duncan Award from Rice University, the RockStar Award (2014) and Test of Time Award (2022) from ACM SIGMOBILE. He is a fellow of ACM.